UNIT-4	MOBILE APPLICATION DEVELOPMENT	3
Android Application Design Essentials: Anatomy of an Android applications, Android terminologies, Application Context, Activities, Services, Intents, Receiving and Broadcasting Intents, Android Manifest File and its common settings, Using Intent Filter, Permissions.		

Unit 4 - **MOBILE APPLICATION DEVELOPMENT**

1.1. Basics of MobileApp Development

Mobile apps are majorly developed for 3 Operating System.

1. Android
2. IOS
3. Windows

Types of Mobile APP

1. Native Apps
Built for a specific platform (Android or iOS) using platform-specific programming languages.
Key Features:
✅ Fast and high performance
✅ Better user experience (UI/UX)
✅ Access to device features (camera, GPS, Bluetooth, etc.)

Examples:
📱 **Android (Java/Kotlin)** – WhatsApp, Instagram
📱 **iOS (Swift/Objective-C)** – Apple Music, iMessage

2. Web Apps
Mobile-friendly websites that run in a browser (Chrome, Safari) instead of being installed.

Key Features:
✅ No need for installation
✅ Cross-platform (runs on any device with a browser)
✅ Easier to maintain
❌ Slower than native apps
❌ Limited access to device features
Examples:
🌐 Gmail Web App
🌐 Twitter Web App

3.Hybrid Apps
A combination of native and web apps – built using web technologies (HTML, CSS, JavaScript) and wrapped in a native shell.
Key Features:
✅ Cross-platform (single codebase for Android & iOS)
✅ Faster development than native apps
✅ Access to some native features
❌ Slower than native apps

❌ UI/UX may not feel fully native

Examples:
🔹 Instagram
🔹 Uber
🔹 LinkedIn
📌 **Built with:** React Native, Flutter, Ionic, Apache Cordova

Cross-Platform Application:

These are frameworks that allow developing total native applications which have access to all the native features of IOS and Android but with the same code base. These apps run on both Android and IOS. So normally the development speeds of these apps are very fast and the maintenance cost is low. The performance speed is comparatively low to 1st party native apps but faster than PWA

Multiple frameworks could be used for cross-platform app development.

- Titanium
- React Native
- Xamarin
- Flutter
- Native Script
- Ionic
- Js
- PhoneGap(Cordova)

Android Application Design Essentials

Android is an **operating system** that is built basically for Mobile phones. It is based on the Linux Kernel and other open-source software and is developed by **Google**. It is used for touchscreen mobile devices such as smartphones and tablets.

But nowadays these are used in Android Auto cars, TV, watches, camera, etc. It has been one of the best-selling OS for smartphones. Android OS was developed by **Android Inc**. which Google bought in 2005.

The app is developed on an application known as Android Studio. These executable apps are installed through a bundle or package called APK(Android Package Kit).

Android Fundamentals

1. Android Programming Languages

In Android, basically, programming is done in two languages **JAVA or C++** and **XML(Extension Markup Language)**. Nowadays KOTLIN is also preferred.

The XML file deals with the design, presentation, layouts, blueprint, etc (as a front-end) while the JAVA or KOTLIN deals with the working of buttons, variables, storing, etc (as a back-end).

Android Components

The App components are the building blocks of Android. Each component has its own role and life cycles i.e. from launching of an app till the end.

Some of these components depend upon others also. Each component has a definite purpose. The four major app components are:

- Activities
- Services
- Broadcast Receivers
- Content Provider

Android Studio

Introduction to Android Studio

Android Studio is the official **Integrated Development Environment (IDE)** for developing Android applications. It is developed by **Google** and based on **IntelliJ IDEA**. Android Studio provides powerful tools for designing, coding, testing, and debugging Android applications.

Features of Android Studio

✅ **Intelligent Code Editor**

- Helps with **code completion, refactoring, and debugging**.
- Supports **Java, Kotlin, and C++** for Android development.

✅ **Flexible Build System (Gradle)**

- Automates the build process and supports **multi-module projects**.
- Allows adding external **dependencies** easily.

✅ **Android Virtual Device (AVD) Manager**

- Creates **virtual devices** (emulators) for testing apps on different screen sizes and versions of Android.

✅ **Real-time Layout Editor**

- Allows designing **UI visually** with drag-and-drop features.
- Supports **previewing layouts on multiple screen sizes**.

✅ **Integrated Firebase Support**

- Connects with **Firebase** for authentication, databases, push notifications, and cloud storage.

✅ **Debugging & Performance Tools**

- Includes **Logcat, Profiler, and Debugger** for monitoring app performance.

UNIT-4	MOBILE APPLICATION DEVELOPMENT	3
Android Application Design Essentials: Anatomy of an Android applications, Android terminologies, Application Context, Activities, Services, Intents, Receiving and Broadcasting Intents, Android Manifest File and its common settings, Using Intent Filter, Permissions.		

Android Application Design Essentials

Android is an **operating system** that is built basically for Mobile phones. It is based on the Linux Kernel and other open-source software and is developed by **Google**. It is used for touchscreen mobile devices such as smartphones and tablets.

But nowadays these are used in Android Auto cars, TV, watches, camera, etc. It has been one of the best-selling OS for smartphones. Android OS was developed by **Android Inc**. which Google bought in 2005.

The app is developed on an application known as Android Studio. These executable apps are installed through a bundle or package called APK(Android Package Kit).

Android Fundamentals

1. **Android Programming Languages**

   In Android, basically, programming is done in two languages **JAVA or C++** and **XML(Extension Markup Language)**. Nowadays KOTLIN is also preferred.

   The XML file deals with the design, presentation, layouts, blueprint, etc (as a front-end) while the JAVA or KOTLIN deals with the working of buttons, variables, storing, etc (as a back-end).

2. **Android Components**

   The App components are the building blocks of Android. Each component has its own role and life cycles i.e. from launching of an app till the end.

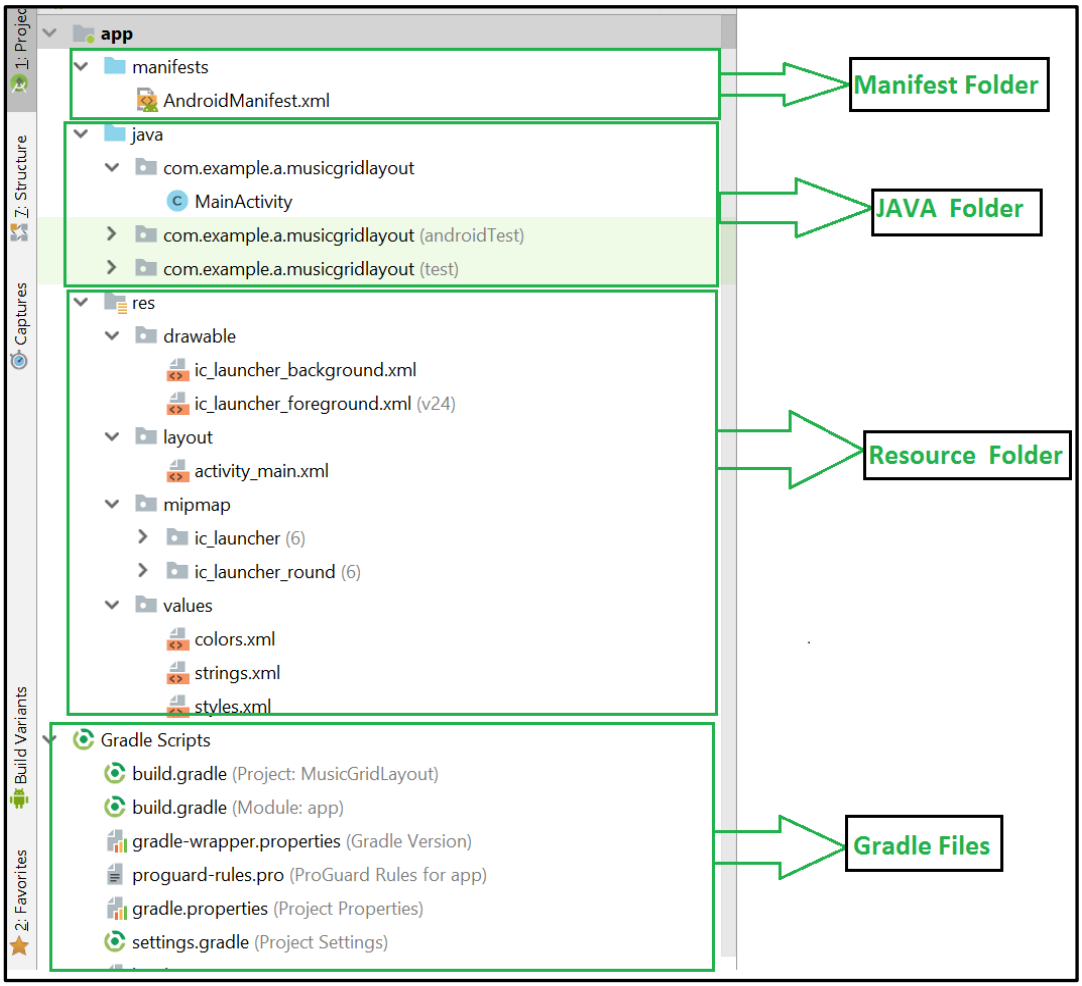
   Some of these components depend upon others also. Each component has a definite purpose. The four major app components are:

   - Activities
   - Services

- Broadcast Receivers
- Content Provider

3. Structural Layout of Android Studio

The basic structural layout of Android Studio is given below:

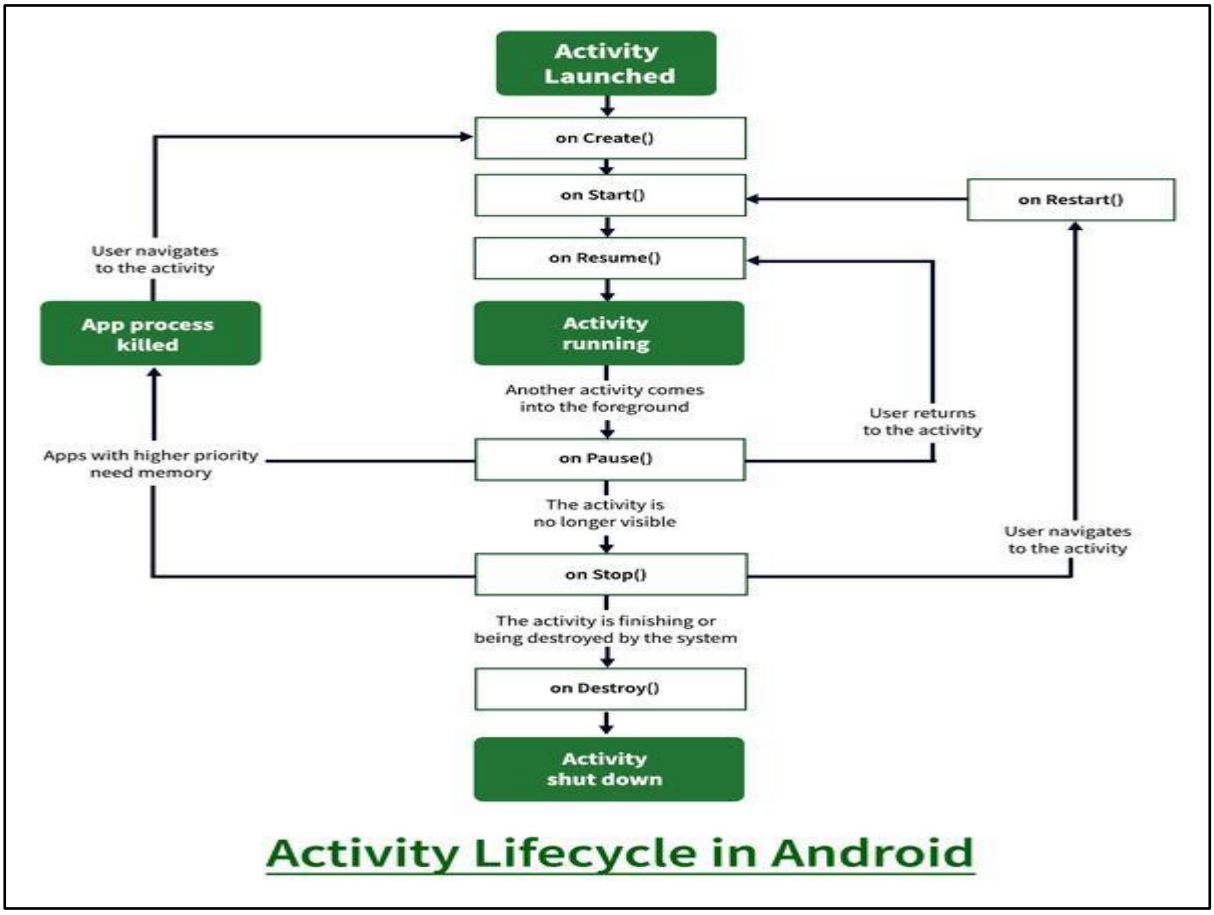


- **Manifest Folder: Android Manifest** is an XML file that is the root of the project source set. It describes the essential information about the app and the Android build tools, the Android Operating System, and Google Play. It contains the permission that an app might need in order to perform a specific task.
- **Java Folder:** The **JAVA folder** consists of the java files that are required to perform the background task of the app. It consists of the functionality of the buttons, calculation, storing, variables, toast(small popup message), programming function, etc.
- **Resource Folder:** The res or Resource folder consists of the various resources that are used in the app. This consists of sub-folders like drawable, layout, mipmap, raw, and values. The layout consists of the XML files that define the user interface layout.

- **Gradle Files**: Gradle is an advanced toolkit, which is used to manage the build process, that allows defining the flexible custom build configurations.

4. Lifecycle of Activity in Android App

The **Lifecycle** of Activity in Android App can be shown through this diagram:



States of Android Lifecycle:

1. **OnCreate:** This is called when activity is first created.
2. **OnStart:** This is called when the activity becomes visible to the user.
3. **OnResume:** This is called when the activity starts to interact with the user.
4. **OnPause:** This is called when activity is not visible to the user.
5. **OnStop:** This is called when activity is no longer visible.
6. **OnRestart:** This is called when activity is stopped, and restarted again.
7. **OnDestroy:** This is called when activity is to be closed or destroyed.

ANATOMY OF AN ANDROID APPLICATIONS

There are some necessary building blocks that an Android application consists of. These loosely coupled components are bound by the application manifest file which contains the description of each component and how they interact. The manifest file also contains the app's metadata, its hardware configuration, and platform requirements, external libraries, and required permissions. There are the following main components of an android app:

1) **Activities**

Activities are said to be the presentation layer of our applications. The UI of our application is built around one or more extensions of the Activity class.

By using Fragments and Views, activities set the layout and display the output and also respond to the user's actions. An activity is implemented as a subclass of class Activity.

```
public class MainActivity extends Activity {
}
```

2) **Services**

Services are like invisible workers of our app. These components run at the backend, updating your data sources and Activities, triggering Notification, and also broadcast Intents.

A service can be used as a subclass of class Service:

```
public class ServiceName extends Service {
}
```

3) **Content Providers**

It is used to manage and persist the application data also typically interacts with the SQL database. They are also responsible for sharing the data beyond the application boundaries. A content provider should be a sub-class of the class ContentProvider.

```
public class contentProviderName extends  ContentProvider {
  public void onCreate(){}
}
```

4) **Broadcast Receivers -** Broadcast Receivers make our application react to any received Intent thereby making them perfect for creating event-driven applications.

ANDROID TERMINOLOGIES

Android SDK (Software Development Kit)

Like JDK of Java, The Android SDK (Software Development Kit) is a set of development tools and libraries provided by Google for creating applications for the Android operating system.

- Android Emulator/Android Virtual Device (AVD)
- Android Debug Bridge (ADB)
- SDK Manager

APK File

An APK file (Android Package Kit) is the package file format used by the Android operating system for the distribution and installation purposes of mobile apps.

An APK file contains all the necessary programming elements, such as the app's code (such as .dex files), resources, assets, certificates, and manifest files, required for installing and running an Android application on a device.

Android Tool Repository

Android Tool Repository' tools are typically available through different sources, including official Google resources, open-source projects, and third-party platforms.

- Android Developer Website:
- Android Open Source Project (AOSP)
- GitHub
- XDA Developers
- Android Studio IDE

Dalvik Virtual Machine (DVM)

Dalvik Virtual Machine (DVM), is a virtual machine used in older versions of the Android operating system to run Android applications.

Manifest File

A "manifest file" generally refers to the "AndroidManifest.xml" file which is an XML file that every Android app must include in its root directory.

Gradle

Gradle is a powerful build system that provides flexibility and efficiency in building Android applications.

Gradle is an advanced build automation tool that is used for building, testing, and deploying Android applications.

Toast

A Toast is a form of short alert message shown on the Android screen for a short interval of time.

Android Toast is actually a short popup notification that is used to display information when we perform any operation/event in our app.

APPLICATION CONTEXT

What is Context?

The Circumstances that form the setting for an Event, Statement, or Idea, and in terms of which it can be fully understood.

In Android Programming Context can be understood as something which gives us the Context of the current state of our application. We can break the Context and its use into three major points:

- It allows us to access resources.
- It allows us to interact with other Android components by sending messages.
- It gives you information about your app environment.

Types of Context in Android

There are mainly two types of Context that are available in Android.

1. **Application Context and**
2. **Activity Context**

Application Context

This Context is tied to the Lifecycle of an Application. Mainly it is an instance that is a singleton and can be accessed via **getApplicationContext().** Some use cases of Application Context are:

- If it is necessary to create a singleton object
- During the necessity of a library in an activity

getApplicationContext():

It is used to return the Context which is linked to the Application which holds all activities running inside it. When we call a method or a constructor, we often have to pass a Context and often we use **"this"** to pass the activity Context or **"getApplicationContext"** to pass the application Context.

Main.java

```
import android.app.Application;
public class GlobalExampleClass extends Application {
    private String globalName;
    private String globalEmail;
    public String getName() {
        return globalName;
    }
    public void setName(String aName) {
        globalName = aName;
    }
    public String getEmail() {
        return globalEmail;
    }
    public void setEmail(String aEmail) {
        globalEmail = aEmail;
    }
}
```

Syntax:

```
// Activity 1
        public class <your activity1> extends Activity {
          ........
          ........
          private <yourapplicationname> globarVar;
          ........
          @Override
          public void onCreate(Bundle savedInstanceState) {
            .......
            final GlobalExampleClass globalExampleVariable =
        (GlobalExampleClass) getApplicationContext();

            // In this activity set name and email and can reuse in other activities
            globalExampleVariable.setName("getApplicationContext example");
            globalExampleVariable.setEmail("xxxxxx@gmail.com");


            .......
        }
```

So, whenever the variable scope is required throughout the application, we can get it by means of **getApplicationContext().** Following is a list of functionalities of Application Context.

List of functionalities of Application Context:

- Load Resource Values
- Start a Service
- Bind to a Service
- Send a Broadcast
- Register BroadcastReceiver

ACTIVITIES

Activity class is one of the very important parts of the Android Component. Most of the programming languages, in which the main() method is the entry point for that program or application to start its execution, the android operating system initiates the code in an Activity instance by invoking specific callback methods that correspond to specific stages of its Lifecycle.

An activity is the entry point for interacting with the user. Every activity contains the layout, which has a user interface to interact with the user.

Layout for a particular activity is set with the help of **setContentView()**. **setContentView()** is a function that takes **View** as a parameter. The view parameter basically contains the layout file for that activity. The code has been given in both Java Programming Language for Android.

```
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Here are two methods almost all subclasses of Activity will implement:

1. **onCreate()**
2. **onPause()**

1. **onCreate() Method**

   The syntax for Java:

   protected void onCreate (Bundle savedInstanceState)

   - **onCreate(Bundle) method** is the place, where the user initializes the activity. This method is called when the activity is starting. This is the method that is used to initialize most of the things in the android app.
   - **onCreate() method** calls the setContentView() method to set the view corresponding to the activity. By default, in any android application, setContentView point to activity_main.xml file, which is the layout file corresponding to MainActivity.

2. **onPause() Method**

   The syntax for Java:

```
protected void onPause() {
    super.onPause();
}
```

   This method is called part of the Activity Lifecycle when the user no longer actively interacts with the activity, but it is still visible on the screen.

INTENTS

Intents, in general, are used for navigating among various activities within the same application, but note, is not limited to one single application, i.e., they can be utilized from moving from one application to another as well.

The process of taking users from one application to another is achieved by passing the Intent to the system.

Intents could be Implicit, for instance, calling intended actions, and explicit as well, such as opening another activity after some operations like onClick or anything else. Below are some applications of Intents:

- Sending the User to Another App
- Getting a Result from an Activity
- Allowing Other Apps to Start Your Activity

Types of Android Intents

There are two types of intents in android

1. Implicit
2. Explicit

1) **Implicit Intent**

Implicit Intent doesn't specify the component. In such a case, intent provides information on available components provided by the system that is to be invoked.

2) **Explicit Intent**

Explicit Intent specifies the component. In such a case, intent provides the external class to be invoked.

Methods in Android Intents

Methods	Description
Context.startActivity()	This is to launch a new activity or get an existing activity to be action.
Context.startService()	This is to start a new service or deliver instructions for an existing service.
Context.sendBroadcast()	This is to deliver the message to broadcast receivers.

RECEIVING AND BROADCASTING INTENTS

Broadcast in android is the system-wide events that can occur when the device starts, when a message is received on the device or when incoming calls are received, or when a device goes to airplane mode, etc.

Broadcast Receivers are used to respond to these system-wide events. Broadcast Receivers allow us to register for the system and application events, and when that event happens, then the register receivers get notified. There are mainly two types of Broadcast Receivers:

- **Static Broadcast Receivers**: These types of Receivers are declared in the manifest file and works even if the app is closed.
- **Dynamic Broadcast Receivers**: These types of receivers work only if the app is active or minimized.

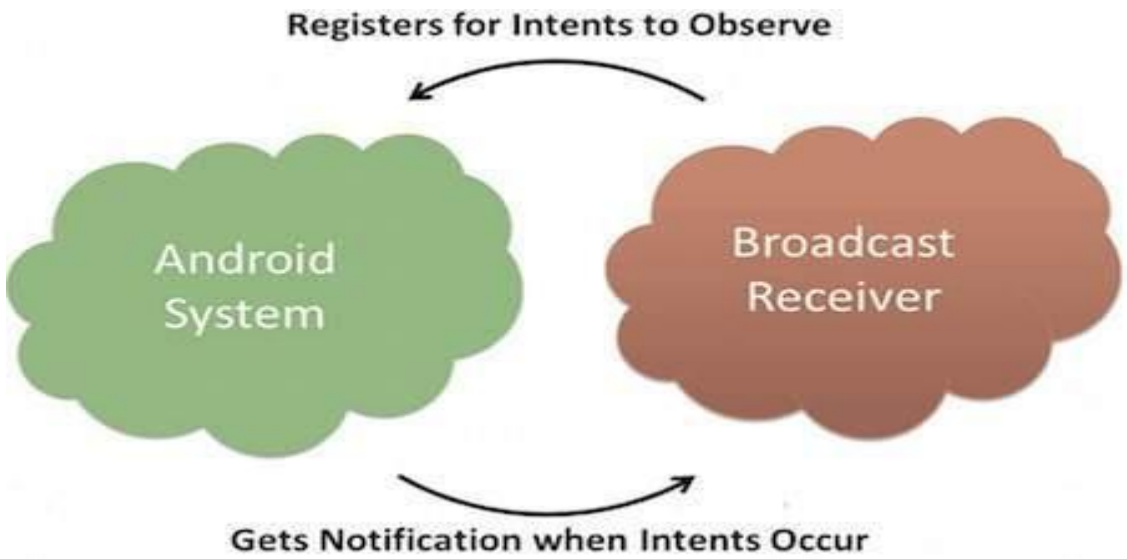
Creating the Broadcast Receiver

A broadcast receiver is implemented as a subclass of BroadcastReceiver class and overriding the onReceive() method where each message is received as a Intent object parameter.

```
public class MyReceiver extends BroadcastReceiver {
  @Override
  public void onReceive(Context context, Intent intent) {
   Toast.makeText(context, "Intent Detected.", Toast.LENGTH_LONG).show();
  }
}
```

Registering Broadcast Receiver

An application listens for specific broadcast intents by registering a broadcast receiver in AndroidManifest.xml file. Consider we are going to register MyReceiver for system generated event ACTION_BOOT_COMPLETED which is fired by the system once the Android system has completed the boot process.



Broadcast-Receiver

```
<application
  android:icon="@drawable/ic_launcher"
  android:label="@string/app_name"
  android:theme="@style/AppTheme" >
  <receiver android:name="MyReceiver">
    <intent-filter>
      <action android:name="android.intent.action.BOOT_COMPLETED">
      </action>
    </intent-filter>
```

```
    </receiver>
</application>
```

ANDROID MANIFEST FILE AND ITS COMMON SETTINGS

Every project in Android includes a Manifest XML file, which is AndroidManifest.xml, located in the root directory of its project hierarchy.

The manifest file is an important part of our app because it defines the structure and metadata of our application, its components, and its requirements.

This file includes nodes for each of the Activities, Services, Content Providers, and Broadcast Receivers that make the application, and using Intent Filters and Permissions determines how they coordinate with each other and other applications.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.example.rec"
    android:versionCode="1"
    android:versionName="1.0"
    android:installLocation="preferExternal">
      <uses-sdk
        android:minSdkVersion="18"
        android:targetSdkVersion="27" />
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication"
```

```
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

A manifest file includes the nodes that define the application components, security settings, test classes, and requirements that make up the application. Some of the manifest sub-node tags that are mainly used are:

1. **manifest**
2. **uses-sdk**
3. **uses-permission**
4. **application**
5. **uses-library**
6. **activity**
7. **intent-filter**
8. **action**
9. **category**
10. **uses-configuration**
11. **uses-features**
12. **permission**

USING INTENT FILTER

Specifies the types of intents that an activity, service, or broadcast receiver can respond to. An intent filter declares the capabilities of its parent component: what an activity or service can do and what types of broadcasts a receiver can handle.

It opens the component to receiving intents of the advertised type while filtering out those that aren't meaningful for the component. Most of the contents of the filter are described by its subelements:

- <action>,
- <category>,
- <data>, and
- <uri-relative-filter-group>.

Example

XML File

```
<intent-filter
  android:icon="drawable resource"
  android:label="string resource"
  android:priority="integer" >
   . . .
</intent-filter>
```

Most of the intent filter are describe by its

1. **<action>,**
2. **<category> and**
3. **<data>.**

1. <action>

Syntax:

XML File

```
<action android:name="string" />
```

Adds an action to an intent filter. An <intent-filter> element must contain one or more <action> elements. If there are no <action> elements in an intent filter, the filter doesn’t accept any Intent objects.

2. <category>

Syntax:

XML File

```
<category android:name="string" />
```

Adds a category name to an intent filter. A string containing additional information about the kind of component that should handle the intent.

3. <data>

Syntax:

XML File

```
<data android:scheme="string"
 android:host="string"
 android:port="string"
 android:path="string"
 android:pathPattern="string"
 android:pathPrefix="string"
 android:mimeType="string" />
```

Adds a data specification to an intent filter. The specification can be just a data type, just a URI, or both a data type and a URI.

PERMISSION

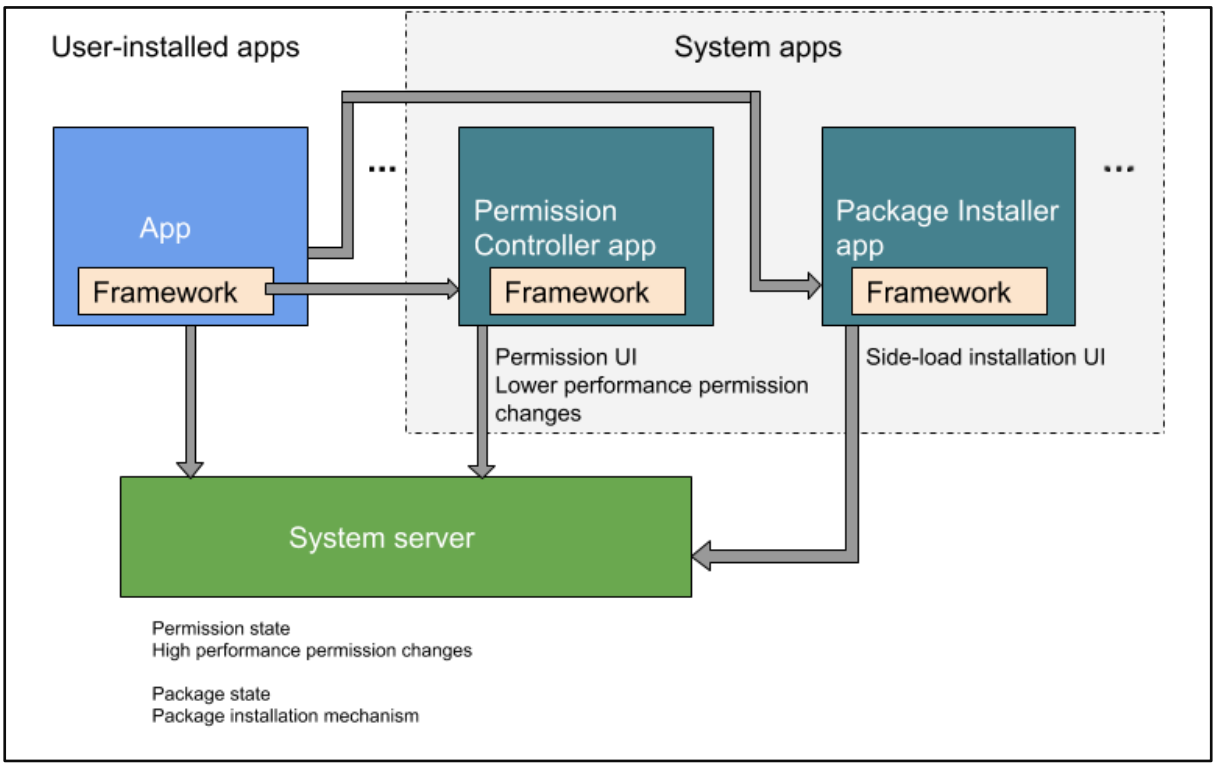
Android permissions provide controls that increase user awareness and limit an app's access to sensitive data. Configuring permissions on Android 8.0 and lower includes allowlisting, without which privileged apps are disabled, even if they're in the priv-app path.

Android 10 introduced the concept of a role, a unique name within the system associated with certain requirements and privileges. Assign roles to apps to grant them permissions for a specific purpose, and configure default roles using platform config resources.

Package install and permission

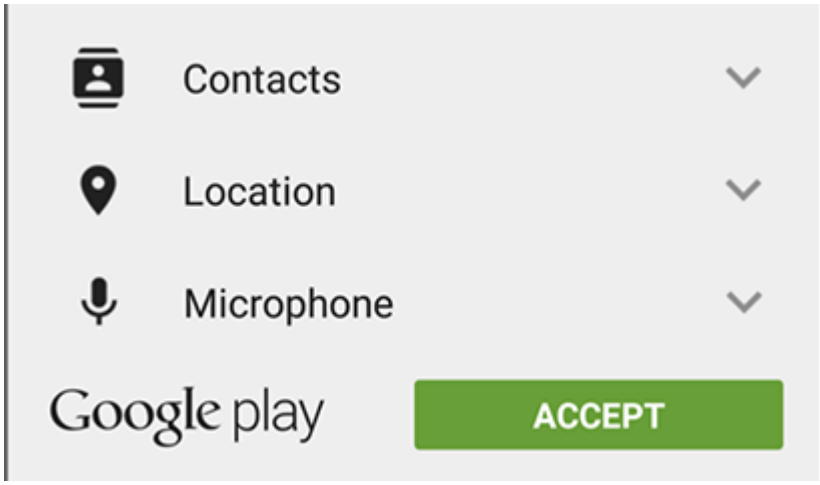
In Android 9 and lower, the package installs and permission control functionalities were contained in the PackageInstaller package

(//packages/apps/PackageInstaller). In Android 10 and higher, the permission control functionality resides in a separate package, PermissionController (//packages/apps/PermissionController). Figure 1 illustrates where the two packages reside in Android 10.



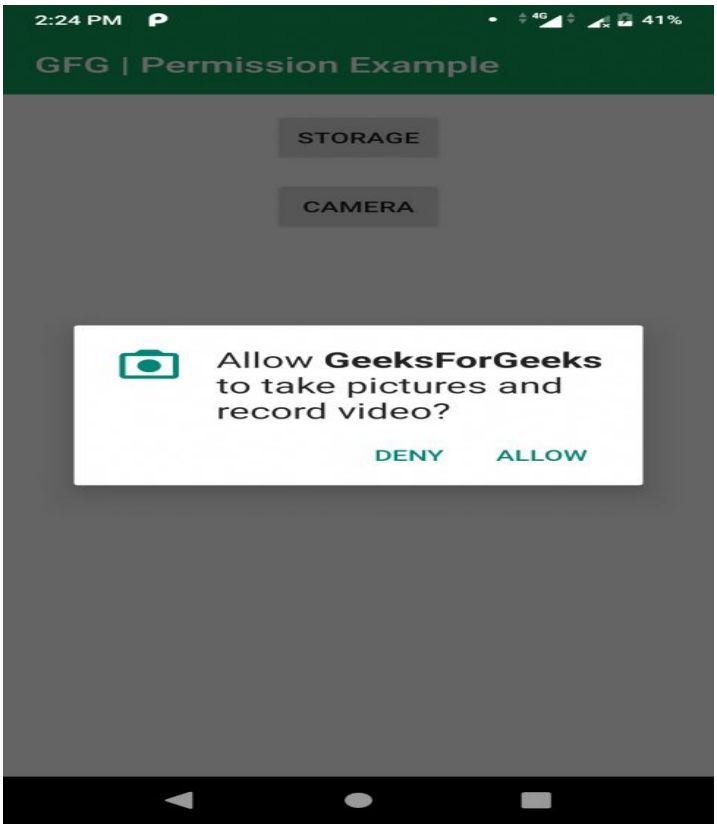
Types of Permissions

1. Install-Time Permissions: If the Android 5.1.1 (API 22) or lower, the permission is requested at the installation time at the Google Play Store.



If the user **Accepts** the permissions, the app is installed. Else the app **installation is canceled**.

2. Run-Time Permissions: If the **Android 6 (API 23) or higher**, the permission is requested at the run time during the running of the app.



- If the user **Accepts** the permissions, then that feature of the app can be used. Else to use the feature, the app **requests permission again**.
- So, now the permissions are requested at runtime. In this article, we will discuss how to request permissions in an Android Application at run time.